

Handleiding Valimo Engine v4

1. Wat Valimo Engine v4 doet

Valimo Engine v4 is de centrale bron voor je designsysteem. In de engine beheer je:

- basistokens zoals spacing, typografie en radius;
- dark- en light-modes;
- accentkleuren;
- semantische tokens;
- herbruikbare componentregels;
- utility classes;
- themes;
- project-specifieke CSS;
- CSS- en JSON-exports.

De engine gebruikt deze volgorde:

```
Core
→ Themes
→ Components
→ Utilities
→ Project-CSS
```

CSS die later wordt geladen, kan eerdere lagen aanvullen of overschrijven.

2. De beschikbare CSS-bestanden

`valimo.css`

Dit is de volledige algemene Valimo-build.

Het bestand bevat:

- core tokens;
- semantic tokens;
- dark- en light-modes;
- alle actieve accents;
- algemene componenten;
- utilities.

Het bevat geen project-specifieke CSS.

Gebruik dit bestand wanneer

Je snel een compleet Valimo-project wilt starten en geen losse frameworkbestanden nodig hebt.

```
<link rel="stylesheet" href="valimo.css">
```

Voor de meeste projecten is dit de eenvoudigste keuze.

Voeg daarna eventueel de projectlaag toe:

```
<link rel="stylesheet" href="valimo.css">
<link rel="stylesheet" href="valimo-project-mijn-project.css">
```

Gebruik niet tegelijkertijd `valimo.css` én de losse core-, theme-, component- en utilitybestanden. Dan laad je dezelfde regels dubbel.

valimo-core.css

Dit bestand bevat de fundering van Valimo:

- fonts;
- tekstgroottes;
- spacing;
- radius;
- layoutwaarden;
- motionwaarden;
- z-indexwaarden;
- semantische aliases;
- resets;
- globale basisregels;
- focus- en accessibilityregels.

Gebruik dit bestand wanneer

Je de frameworklagen afzonderlijk wilt laden.

```
<link rel="stylesheet" href="valimo-core.css">
```

`valimo-core.css` is niet bedoeld om zelfstandig een volledig vormgegeven project te leveren. De mode- en accentwaarden ontbreken dan nog.

valimo-themes.css

Dit bestand bevat:

- dark mode;
- light mode;
- alle actieve accentkleuren;
- surface-opacitywaarden;
- accentvariabelen;
- contrastkleur voor tekst op accentknoppen.

```
<link rel="stylesheet" href="valimo-themes.css">
```

Het bestand verwacht dat `valimo-core.css` ervoor geladen is.

Correct:

```
<link rel="stylesheet" href="valimo-core.css">
<link rel="stylesheet" href="valimo-themes.css">
```

Je activeert een mode en accent op het `<html>`-element:

```
<html lang="nl" data-theme="dark" data-accent="green">
```

Voor een lichte blauwe versie:

```
<html lang="nl" data-theme="light" data-accent="blue">
```

Je kunt dit ook met JavaScript wijzigen:

```
document.documentElement.dataset.theme = 'light';
document.documentElement.dataset.accent = 'purple';
```

Belangrijk over theme-records

De engine bevat ook records zoals:

```
data-valimo-theme="valimo-default"
```

In de huidige v4-export legt zo'n record alleen vast welke mode en accent erbij horen. Het record zet die waarden nog niet automatisch door naar `data-theme` en `data-accent`.

Gebruik daarom in je project voorlopig altijd expliciet:

```
<html
  data-valimo-theme="valimo-default"
  data-theme="dark"
  data-accent="green"
>
```

`data-theme` en `data-accent` bepalen daadwerkelijk de kleuren.

`valimo-components.css`

Dit bestand bevat de algemene Valimo-componenten, bijvoorbeeld:

- buttons;
- panels;
- cards;
- inputs;
- navigatie-items;
- badges en signals;
- tabellen;
- modals;
- toasts;
- statusregels;
- field groups.

Gebruik het na core en themes:

```
<link rel="stylesheet" href="valimo-core.css">
<link rel="stylesheet" href="valimo-themes.css">
<link rel="stylesheet" href="valimo-components.css">
```

Voorbeeld:

```
<div class="panel">
  <h2>Instellingen</h2>

  <div class="field-group">
    <label class="field-label" for="project-name">Projectnaam</label>
    <input id="project-name" value="Mijn project">
    <p class="field-hint">Een korte herkenbare naam.</p>
  </div>

  <button class="btn primary">Opslaan</button>
</div>
```

valimo-utilities.css

Dit bestand bevat kleine herbruikbare utility classes.

Voorbeelden:

```
<div class="flex items-center justify-between gap-3">
  ...
</div>
```

```
<div class="surface-2 border radius-1 p-4">
  ...
</div>
```

De utilities gebruiken de tokens uit core en themes. Daarom moet je ze niet zelfstandig laden.

Correcte volgorde:

```
<link rel="stylesheet" href="valimo-core.css">
<link rel="stylesheet" href="valimo-themes.css">
<link rel="stylesheet" href="valimo-components.css">
<link rel="stylesheet" href="valimo-utilities.css">
```

Wanneer utilities gebruiken

Gebruik utilities voor kleine compositiekeuzes:

- padding;
- margin;
- gap;
- flex of grid;
- alignment;
- tekstkleur;
- oppervlak;
- border;
- radius;
- eenvoudige states.

Gebruik utilities niet voor een volledig projectspecifiek component met veel gedrag.

Goed:

```
<div class="flex items-center gap-2 p-3 surface-2">
```

Minder geschikt:

```
<div class="player-card-with-score-and-market-status">
```

Dat laatste hoort als projectcomponent in Project-CSS.

valimo-project-<slug>.css

Dit bestand bevat CSS die alleen bij één project hoort.

Voorbeelden:

- een voetbalveld;
- spelerskaarten;
- een EPG-grid;
- een videoplayer;
- een specifieke dashboardlayout;
- projectgebonden IDs;
- selectors die door project-JavaScript worden gebruikt;
- uitzonderingen voor één website.

Voorbeeld:

```
<link rel="stylesheet" href="valimo.css">  
<link rel="stylesheet" href="valimo-project-voetbal-analyser.css">
```

De projectlaag moet altijd als laatste worden geladen:

```
<!-- Goed -->  
<link rel="stylesheet" href="valimo.css">  
<link rel="stylesheet" href="valimo-project-klantenportaal.css">
```

Niet andersom:

```
<!-- Verkeerd -->  
<link rel="stylesheet" href="valimo-project-klantenportaal.css">  
<link rel="stylesheet" href="valimo.css">
```

Anders kan de basis de projectaanpassingen weer overschrijven.

valimo-tokens.json

Dit bestand bevat de systeemgegevens in JSON-vorm:

- foundations;

- modes;
- accents;
- semantic tokens;
- themes;
- components;
- utilities;
- presets;
- projectlaagmetadata.

Gebruik dit bestand wanneer

Je tokens buiten CSS nodig hebt, bijvoorbeeld voor:

- JavaScript;
- een eigen buildtool;
- documentatie;
- een design-tokenviewer;
- koppeling met andere systemen;
- automatische generatie van configuratie.

Voor een normaal HTML/PHP-project hoeft je dit bestand meestal niet te laden.

3. Welke exportmethode moet je gebruiken?

Op de exportpagina zijn twee verschillende mogelijkheden.

Direct downloaden

De downloadknoppen genereren het gekozen bestand direct vanuit de database.

Voorbeelden:

- Download `valimo.css`;
- Core;
- Themes;
- Components;
- Utilities;
- Tokens JSON.

Gebruik direct downloaden wanneer je één bestand nodig hebt.

Er wordt dan niets permanent in de map van de engine opgeslagen. De inhoud wordt direct als download aangeboden.

Volledige export genereren

De knop **Genereer volledige export** schrijft alle bestanden naar dezelfde servermap als de engine:

```
valimo.css
valimo-core.css
valimo-themes.css
valimo-components.css
valimo-utilities.css
valimo-tokens.json
valimo-project-*.css
```

De export wordt daarnaast geregistreerd in de exporttabel van de database.

Gebruik dit wanneer:

- de engine en je project op dezelfde server staan;
- je alle bestanden in één keer wilt verversen;
- je de gegenereerde bestanden via deployment wilt kopiëren;
- je een vaste exportmap gebruikt.

De PHP-map moet hiervoor schrijfrechten hebben.

4. Aanbevolen gebruik per project

Eenvoudigste werkwijze

Gebruik voor een normaal project:

```
<link rel="stylesheet" href="/css/valimo.css">
<link rel="stylesheet" href="/css/valimo-project-mijn-project.css">
```

En stel het theme in:

```
<html lang="nl" data-theme="dark" data-accent="green">
```

Dit is de aanbevolen standaard.

Modulaire werkwijze

Gebruik losse bestanden wanneer je meer controle over caching of bundling wilt:

```
<link rel="stylesheet" href="/css/valimo-core.css">
<link rel="stylesheet" href="/css/valimo-themes.css">
<link rel="stylesheet" href="/css/valimo-components.css">
<link rel="stylesheet" href="/css/valimo-utilities.css">
<link rel="stylesheet" href="/css/valimo-project-mijn-project.css">
```


Gebruik óf `valimo.css`, óf de vier losse algemene bestanden.

Niet beide.

5. Themes gebruiken

Open het tabblad **Themes**.

Een theme bestaat uit:

- slug;
- naam;
- mode;
- accent;
- status;
- beschrijving;
- eventueel de aanduiding standaardtheme.

Voorbeeld:

```
Naam: Klantenportaal Green
Slug: klantenportaal-green
Mode: dark
Accent: green
Status: active
```

Statussen

Draft

Gebruik `draft` voor een theme waaraan je nog werkt.

Draftthemes worden momenteel wel opgenomen in de theme-export zolang ze niet archived zijn. De status is dus vooral administratief.

Active

Gebruik `active` voor themes die klaar zijn voor productie.

Archived

Gebruik `archived` voor oude themes die je wilt bewaren, maar niet meer in de CSS-export wilt opnemen.

Archived themes worden niet in `valimo-themes.css` gezet.

Theme dupliceren

Gebruik dupliceren wanneer je een bestaand theme als uitgangspunt wilt nemen.

Voorbeeld:

```
Bron: valimo-default  
Nieuwe slug: klantenportaal-v2  
Nieuwe naam: Klantenportaal v2
```

De kopie wordt als `draft` opgeslagen.

Daarna kun je de gekoppelde mode en accent aanpassen.

6. Modes gebruiken

Modes bepalen de neutrale basis:

- achtergrond;
- alternatieve achtergrond;
- tekst;
- muted tekst;
- zachte tekst;
- borders;
- surface-opacity 1 tot en met 4.

Voorbeelden:

```
dark  
light
```

Een mode bevat bewust geen specifieke accentkleur.

Gebruik modes voor structurele verschillen tussen licht en donker, niet voor projectkleuren.

Wanneer een nieuwe mode maken?

Maak alleen een nieuwe mode wanneer de volledige neutrale basis anders moet zijn.

Bijvoorbeeld:

```
dark  
light  
high-contrast-dark
```

Maak geen aparte mode voor iedere accentkleur.

Dus niet:

```
green-dark  
blue-dark  
red-dark
```

Daarvoor zijn accents bedoeld.

7. Accents gebruiken

Accents bepalen de signaalkleur:

```
• accent ;  
• accent-2 ;  
• accent-rgb ;  
• accent-ink ;  
• accent-line ;  
• accent-contrast .
```

Accentkleur wordt gebruikt voor betekenisvolle onderdelen, zoals:

- primary buttons;
- actieve navigatie;
- focus;
- iconen;
- actieve borders;
- geselecteerde states.

Nieuw accent maken

Open **Accents** en vul bijvoorbeeld in:

```
Slug: petrol  
Naam: Petrol  
Accent: #496f73  
Accent 2: #719397  
Ink opacity: 0.14  
Line opacity: 0.28
```

Je kunt de engine ook een lichtere tweede accentkleur laten afleiden.

Controleer daarna altijd de preview en contrastcontrole.

8. Tokens aanpassen

Open het tabblad **Tokens**.

Er zijn twee soorten tokens.

Foundation tokens

Dit zijn ruwe systeemwaarden:

```
space-1  
space-4  
radius-1  
font-body  
font-heading  
text-sm  
transition-fast
```

Gebruik deze voor waarden die door het hele systeem gedeeld worden.

Voorbeeld:

```
Token: radius-1  
Waarde: 2px  
Categorie: shape
```

Wijzig je `radius-1`, dan veranderen alle componenten die dit token gebruiken.

Semantic tokens

Semantische tokens beschrijven een functie:

```
color-page  
color-panel  
color-control  
color-text-primary  
color-action-primary  
color-focus
```

Voorbeeld:

```
--color-panel: var(--surface-1);
```

Componenten gebruiken bij voorkeur semantic tokens.

Dat is beter dan in ieder component rechtstreeks `--surface-1` of een hexkleur te gebruiken.

9. Componenten beheren

Open **Componenten**.

Een componentregel bevat:

- key;
- selector;
- CSS-block;
- scope;
- volgorde;
- enabled-status.

Voorbeeld:

```
Key: toolbar
Selector: .toolbar
Scope: component
CSS block:
display: flex;
align-items: center;
gap: var(--space-3);
```

Scope `core`

Gebruik core alleen voor globale regels zoals:

- reset;
- body;
- headings;
- focus;
- selection;
- reduced motion.

Core-regels komen in `valimo-core.css`.

Scope `component`

Gebruik dit voor algemeen herbruikbare Valimo-componenten:

- buttons;
- cards;
- modals;
- inputs;
- nav-items;

- badges;
- tabellen.

Deze komen in `valimo-components.css`.

Scope `project`

Projectregels kun je technisch als component opslaan, maar voor duidelijke scheiding is het beter om echte projectspecifieke CSS onder **Project-CSS** te bewaren.

10. Utilities beheren

Open **Utilities**.

Een utility bestaat uit:

- key;
- selector;
- categorie;
- sort order;
- CSS-block;
- enabled-status.

Voorbeeld:

```
Key: stack-4
Selector: .stack-4
Categorie: layout
CSS:
display:grid;
gap:var(--space-4);
```

Daarna kun je in je project gebruiken:

```
<div class="stack-4">
  ...
</div>
```

Goede utilities

```
.p-4
.gap-3
.flex
.grid
.items-center
```

```
.justify-between  
.surface-2  
.text-muted  
.radius-1  
.border  
.hidden
```

Geen goede utility

```
.customer-dashboard-special-header
```

Dat is geen kleine compositieklasse maar een projectcomponent. Zet die bij Project-CSS.

11. Project-CSS toevoegen

Open **Project-CSS**.

Je kunt:

- een `.css`-bestand uploaden;
- CSS plakken;
- naam en slug instellen;
- een bronnaam toevoegen;
- notities opslaan;
- aangeven of de laag mee moet in de export.

Voorbeeld:

```
Slug: epg-player  
Naam: EPG Player  
Bronnaam: epg.css  
Notities: Tijdlijn, videoplayer en channel cards
```

De export heet dan:

```
valimo-project-epg-player.css
```

Wanneer Project-CSS gebruiken?

Gebruik Project-CSS wanneer een regel afhankelijk is van:

- specifieke HTML uit één project;
- JavaScript-selectors;
- specifieke IDs;

- een externe library;
- projectfunctionaliteit;
- domeinspecifieke componenten.

Voorbeelden:

```
#videoPlayer
.epg-row
.player-card
.pitch
.market-results
.admin-layout
```

Projectlaag uitschakelen

Haal het vinkje **Meenemen in export** weg.

De CSS blijft in de database staan, maar wordt niet meegenomen bij een volledige export.

Via de directe projectdownload kan het bestand in de huidige versie nog steeds handmatig worden gedownload. Het vinkje bepaalt vooral deelname aan **Genereer volledige export**.

12. CSS importeren

Open **Slimme import**.

Plak bestaande CSS in het veld en geef de import een herkenbare naam.

Voorbeeld:

```
:root {
  --space-custom: 18px;
  --brand-special: #5c7a63;
}

.stack-custom {
  display: grid;
  gap: var(--space-custom);
}

.customer-panel {
  padding: 20px;
}
```

Klik op **Importeren**.

Wat de importer doet

De importer:

1. zoekt custom properties;
2. slaat gevonden variabelen op als foundation tokens;
3. leest CSS-blocks;
4. probeert iedere selector te classificeren;
5. zet regels in Import staging;
6. verandert componenten en utilities nog niet automatisch.

De mogelijke detecties zijn:

```
theme
utility
component
project
```

Theme-detectie

Selectors zoals:

```
:root
[data-theme="dark"]
[data-theme="light"]
[data-accent="green"]
```

worden als theme herkend.

De variabelen binnen de import worden automatisch als foundation tokens opgeslagen.

Let hierbij op: de huidige importer zet geïmporteerde custom properties niet automatisch in de tabellen `modes` of `accents`. Controleer na import daarom bij **Tokens** welke waarden zijn toegevoegd en zet echte mode- of accentwaarden eventueel handmatig in Modes of Accents.

Utility-detectie

Klassen die lijken op:

```
.p-4
.gap-2
.text-muted
.surface-2
.border-strong
```

```
.flex-row  
.grid-2
```

worden waarschijnlijk als utility herkend.

Project-detectie

Selectors met bijvoorbeeld:

```
#een-id  
.player  
.pitch  
.epg  
.timeline  
.market  
.admin  
.stream  
.channel
```

worden waarschijnlijk als projectregel herkend.

Component-detectie

Algemene selectors die niet duidelijk utility of project zijn, worden als component gezien.

13. Import staging gebruiken

Na import verschijnen de gevonden regels in **Import staging**.

Per regel zie je:

- importnaam;
- selector;
- automatische detectie;
- of de regel al verwerkt is;
- de gewenste doellaag.

Je kiest vervolgens:

```
component  
utility  
project
```

Naar component

Kies `component` voor algemeen herbruikbare UI.

Voorbeelden:

```
.toolbar  
.empty-state  
.dialog-header  
.data-table
```

Naar utility

Kies `utility` voor één kleine taak.

Voorbeelden:

```
.stack-4  
.text-soft  
.surface-active
```

Naar project

Kies `project` voor regels die alleen bij één project horen.

Voorbeelden:

```
#videoPlayer  
.epg-program-block  
.player-card
```

Na klikken op **Accepteer** wordt de regel opgeslagen in de gekozen laag en als verwerkt gemarkeerd.

Belangrijke beperking

Wanneer je losse geïmporteerde regels als project accepteert, maakt v4 per selector een automatisch projectrecord met een gegenereerde slug.

Voor een groot bestaand projectbestand is het daarom meestal overzichtelijker om:

1. algemene tokens en componenten via Slimme import te verwerken;
2. het volledige projectspecifieke restant als één bestand via Project-CSS te uploaden.

14. Aanbevolen importworkflow

Gebruik voor een oud project deze volgorde.

Stap 1: maak een back-up

Bewaar het originele CSS-bestand.

Stap 2: importeer de CSS via Slimme import

Laat de engine de tokens en selectors analyseren.

Stap 3: controleer Tokens

Bekijk welke variabelen zijn toegevoegd.

Verplaats conceptueel:

- algemene spacing naar foundations;
- algemene kleuren naar modes of accents;
- rolgebonden kleuren naar semantic tokens.

Stap 4: controleer Import staging

Zet alleen echt algemene regels in Components of Utilities.

Stap 5: maak een opgeschoonde projectlaag

Plaats de resterende projectregels als één bestand onder Project-CSS.

Stap 6: exporteer

Download:

```
valimo.css  
valimo-project-<project>.css
```

Stap 7: test in je project

Laad de Valimo-basis eerst en project-CSS als laatste.

Controleer:

- layout;
- buttons;
- inputfocus;

- actieve states;
- light en dark;
- responsive gedrag;
- JavaScript-selectors.

15. Voorbeeld van een nieuw project

HTML

```
<!doctype html>
<html lang="nl" data-theme="dark" data-accent="green">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <link rel="stylesheet" href="/css/valimo.css">
  <link rel="stylesheet" href="/css/valimo-project-dashboard.css">

  <title>Dashboard</title>
</head>
<body>
  <main class="p-5">
    <section class="panel">
      <div class="panel-header">
        <div>
          <span class="signal accent">Actief</span>
          <h1>Dashboard</h1>
        </div>

        <button class="btn primary">Nieuwe invoer</button>
      </div>

      <div class="grid gap-4">
        <div class="surface-2 p-4">
          Inhoud
        </div>
      </div>
    </section>
  </main>
</body>
</html>
```

JavaScript themewissel

```
function setTheme(mode, accent) {
  document.documentElement.dataset.theme = mode;
```

```
document.documentElement.dataset.accent = accent;
}

setTheme('dark', 'green');
```

16. Wat moet je downloaden?

Je start een normaal project

Download:

```
valimo.css
valimo-project-<jouw-project>.css
```

Je project heeft geen eigen CSS nodig

Download alleen:

```
valimo.css
```

Je gebruikt een bundler of losse frameworklagen

Download:

```
valimo-core.css
valimo-themes.css
valimo-components.css
valimo-utilities.css
valimo-project-<jouw-project>.css
```

Je wilt tokens in JavaScript of tooling gebruiken

Download daarnaast:

```
valimo-tokens.json
```

Je hebt wijzigingen in de engine gemaakt

Download of genereer de exports opnieuw.

CSS-bestanden die je eerder hebt gedownload veranderen namelijk niet automatisch mee.

17. Wanneer opnieuw exporteren?

Exporteer opnieuw wanneer je iets wijzigt aan:

- tokens;
- modes;
- accents;
- components;
- utilities;
- project-CSS;
- themeconfiguratie.

Alleen wijzigingen in administratieve beschrijvingen hoeven niet altijd een nieuwe CSS-export op te leveren.

18. Aanbevolen dagelijkse werkwijze

1. Kies of maak een mode.
 2. Kies of maak een accent.
 3. Maak een theme-record.
 4. Controleer de preview.
 5. Pas foundations en semantic tokens alleen aan wanneer de wijziging systeemwijd geldt.
 6. Plaats algemene UI in Components.
 7. Plaats kleine compositieklassen in Utilities.
 8. Plaats projectspecifieke functies in Project-CSS.
 9. Download `valimo.css`.
 10. Download de juiste projectlaag.
 11. Test dark, light en relevante accents.
 12. Exporteer opnieuw na iedere definitieve wijziging.
-

19. Snelle beslisregel

Vraag bij iedere CSS-regel:

Geldt dit voor alle Valimo-projecten?

Ja:

- token;
- component;
- utility.

Nee:

- Project-CSS.

Is het één kleine visuele of layouttaak?

Ja:

- Utility.

Is het een herkenbaar herbruikbaar UI-element?

Ja:

- Component.

Is het een ruwe systeemwaarde?

Ja:

- Foundation token.

Beschrijft het de functie van een kleur of oppervlak?

Ja:

- Semantic token.

Verandert het tussen dark en light?

Ja:

- Mode.

Is het de wisselbare signaalkleur?

Ja:

- Accent.

20. Aanbevolen standaard voor jouw projecten

Gebruik standaard deze combinatie:

```
<html data-theme="dark" data-accent="green">
<head>
  <link rel="stylesheet" href="valimo.css">
  <link rel="stylesheet" href="valimo-project-projectnaam.css">
</head>
```


Gebruik losse frameworkbestanden alleen wanneer je daar technisch een reden voor hebt.

De algemene regel is:

Valimo bepaalt de visuele grammatica.
Het theme bepaalt mode en accent.
Utilities bouwen kleine composities.
Components leveren herbruikbare UI.
Project-CSS levert de specifieke functionaliteit.